

# ทำไม Risk Score ถึงออกมาเป็นเลขนี้

ไล่โค้ด scripts/build\_risk\_points.py ทั้งไฟล์ ฟังก์ชันต่อฟังก์ชัน ตั้งแต่  
โหลด Excel จนถึงจุดสี่ที่ขึ้นบนแผนที่จริง — สูตรทุกเกณฑ์ ตัวแปรทุกตัว เหตุผลของทุก  
ทางเลือก และตัวอย่างจริงจากข้อมูล 171 จุดเสี่ยงที่คำนวณไว้แล้ว

ไฟล์ต้นทาง data/accident2025\_1.xlsx — 4,627 เหตุการณ์ · 6 จังหวัด

สคริปต์ scripts/build\_risk\_points.py

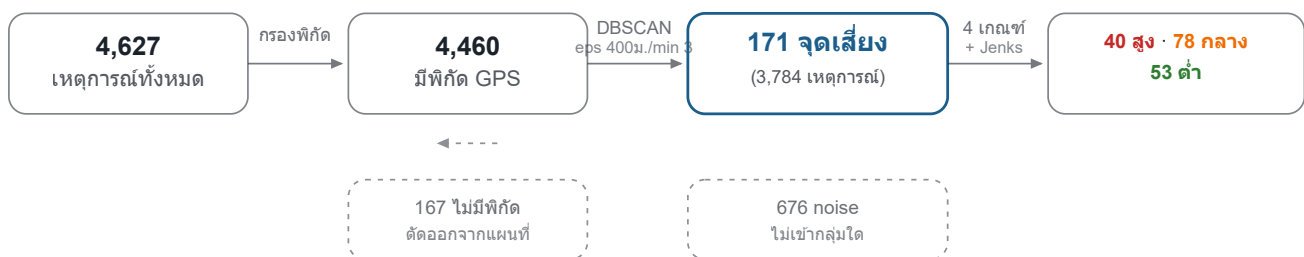
รอบ calibration v2568-r2 (ล็อกแล้ว)

DBSCAN eps = 400 ม. · min\_samples = 3

ผลลัพธ์ 171 จุดเสี่ยง — สูง 40 · ปานกลาง 78 · ต่ำ 53

## 00 ภาพรวมทั้งระบบ

สคริปต์นี้ไม่ได้คำนวณสด ๆ ทุกครั้งที่มีคนเปิดเว็บ — มันรันเป็นรอบ (ทุก 6 เดือน) แล้ว "ล็อก" ผลลัพธ์เป็นไฟล์ ก่อนจะพุดถึงสูตรใด ๆ ต้องเห็นภาพรวมทั้งสายพานก่อนว่าข้อมูลไหลจากอะไรไปอะไร



สายพานทั้งหมดของ build\_risk\_points.py — ตัวเลขจริงจากรอบ calibration v2568-r2

ไฟล์เดียวทำ 5 ขั้นตอนเรียงกัน (ตามที่เขียนไว้ใน docstring บนสุดของไฟล์):

1. โหลด Excel 6 ชีต (1 ชีตต่อจังหวัด) รวมเป็นชุดเดียว
2. จัดกลุ่ม เหตุการณ์ที่อยู่ใกล้กันด้วย DBSCAN ให้กลายเป็น "จุดเสี่ยง" 1 จุด
3. คำนวณ 4 เกณฑ์ ต่อจุด แล้วรวมเป็น Risk Score เดียว 0–100
4. แบ่ง 3 ระดับ ด้วย Jenks Natural Breaks
5. บันทึก เป็น GeoJSON + เก็บ snapshot ของรอบ calibration นี้ไว้ตรวจสอบย้อนหลัง

ทุกหัวข้อถัดจากนี้เดินตามลำดับนี้เป๊ะ ๆ — แต่ละหัวข้อจะโชว์โค้ดจริงของฟังก์ชันที่เกี่ยวข้อง อธิบายทีละบรรทัดว่าทำหน้าที่อะไร แล้วค่อยอธิบายหลักการ/สูตรที่อยู่เบื้องหลัง

## 01 โหลด + ทำความสะอาดข้อมูล

3 ฟังก์ชัน: `load_excel_records`, `clean_points`, `_event_from_record`

`load_excel_records(path)`

รวม 6 ชีต

```
xl = pd.ExcelFile(path)
frames = []
for sheet in xl.sheet_names:
    df = xl.parse(sheet)
    frames.append(df)
merged = pd.concat(frames, ignore_index=True)
return merged.to_dict(orient="records")
```

`xl.sheet_names` คือชื่อชีตทั้ง 6 ในไฟล์ (1 ชีตต่อ 1 จังหวัด — กรุงเทพฯ, นนทบุรี, ปทุมธานี, สมุทรปราการ, นครปฐม, สมุทรสาคร) ลูป `for` อ่านทีละชีตด้วย `xl.parse(sheet)` ได้ตารางข้อมูล (DataFrame) แยกกัน 6 ตาราง เก็บไว้ในลิสต์ `frames`

`pd.concat(frames)` เอาทั้ง 6 ตารางมาต่อกันเป็นตารางเดียวยาว ๆ (`ignore_index=True` คือรีเซ็ตเลขแถวใหม่ตั้งแต่ 0 ไม่ให้เลขแถวซ้ำกันข้ามชีต) แล้ว `.to_dict(orient="records")` แปลงตารางเป็นลิสต์ของ dict — 1 dict ต่อ 1 แถว/1 อุบัติเหตุ เพื่อให้โค้ดถัดไปวนลูปด้วย Python ธรรมดาได้ง่าย ๆ

```
clean_points(records) · _event_from_record(r, lat, lng)
```

กรอง + ทำความสะอาด

```
for r in records:
    if r.get("จังหวัด") not in BANGKOK_METRO_PROVINCES:
        continue
    try:
        lat = float(r["LATITUDE"]); lng = float(r["LONGITUDE"])
    except (TypeError, ValueError, KeyError):
        continue
    if not lat or not lng or np.isnan(lat) or np.isnan(lng):
        continue
```

วนทุกแถว เก็บเฉพาะแถวที่ (1) จังหวัดอยู่ใน 6 จังหวัดเป้าหมาย และ (2) แปลงค่า LATITUDE/LONGITUDE เป็นตัวเลขได้จริง (ไม่ใช่ช่องว่างหรือตัวอักษร) และ (3) ค่านั้นไม่ใช่ 0 และไม่ใช่ NaN (Not a Number — ช่องที่ไม่มีข้อมูล) แถวไหนตกเงื่อนไขข้อใดข้อหนึ่ง โดน `continue` ข้ามไปทันที ไม่เข้ากระบวนการต่อ

แถวที่รอดจะถูกส่งเข้า `_event_from_record` ซึ่งข้างในมี 2 ฟังก์ชันช่วยเล็ก ๆ: `_int(v)` พยายามแปลงเป็นจำนวนเต็มไม่ติดลบ ถ้าแปลงไม่ได้ (ช่องว่าง/ข้อความ) ให้เป็น 0 แทนการพัง และ `_text(v)` แปลงเป็นข้อความที่ตัดช่องว่างหัวท้ายแล้ว ถ้าเป็นค่าว่าง/NaN ให้เป็น "ไม่ระบุ" แทน — ทั้งสองฟังก์ชันนี้มีไว้กันข้อมูล Excel ที่ไม่สมบูรณ์ (ช่องว่าง, พิมพ์ผิด) ไม่ให้ทำให้โปรแกรมพังกลางทาง

ผลลัพธ์คือ dict สะอาด 1 ใบต่อ 1 อุบัติเหตุ มีคีย์ที่ใช้ได้ตลอดทั้งไฟล์: `lat`, `lng`, `province`, `road`, `cause`, `location_type`, `road_type`, `crash_pattern`, `vehicles`, `deaths`, `serious`, `minor`

จากข้อมูลจริง: 4,627 เหตุการณ์ทั้งหมด → เหลือ 4,460 แถวที่ผ่านทั้ง 3 เงื่อนไข (167 แถวไม่มีพิกัดใช้งานได้  $\approx 3.6\%$  ถูกตัดออกจากขั้นตอนจับกลุ่ม — แต่ยังถูกใช้คำนวณน้ำหนักกายภาพถนนในหัวข้อ 04 เพราะเรื่องนั้นไม่ต้องใช้พิกัด)

## 02 DBSCAN จับกลุ่มจุดเสียง — จากศูนย์

ทำไมต้องจัดกลุ่มก่อน? เพราะอุบัติเหตุ "จุดเดียวกันในชีวิตจริง" ถูกบันทึกผิดพลาด

กระจัดกระจายเล็กน้อยเสมอ (คลาดเคลื่อนจาก GPS, จุดตัดถนนช่วงยาว ฯลฯ) ถ้าเอาทุก

แถวมาคิดคะแนนแยกกันเป็นจุด ๆ จะได้จุดกระจายเป็นพัน ๆ จุดที่ไม่มีมีความหมายเชิงพื้นที่

ต้องรวมเหตุที่เกิด "ในย่านเดียวกัน" ให้เป็น 1 จุดเสียงก่อน แล้วค่อยให้คะแนนจุดนั้น

ทำไมเป็น DBSCAN ไม่ใช่วิธีอื่น

ไม่รู้ล่วงหน้าว่ากรุงเทพฯ+ปริมณฑลทั้งหมดควรมี "จุดเสียง" กี่จุด — วิธีอย่าง K-means ต้อง

กำหนดจำนวนกลุ่ม (k) ล่วงหน้า ซึ่งไม่มีทางรู้ตัวเลขที่ถูกต้อง และจุดเสียงจริงมักเรียงยาวตามแนว

ถนน (โค้ง, สะพาน) เป็นรูปทรงยาวรี ไม่ใช่ช่วงกลมเรียบ ๆ — K-means จะบังคับแบ่งเป็นกลุ่มทรง

กลม/นูนเท่านั้น และที่สำคัญคือต้องแยก "เหตุกระจัดกระจายเดี่ยว ๆ" ออกจาก "แบล็กสปอตจริง"

ได้เอง ซึ่ง DBSCAN มีแนวคิดเรื่อง "noise" ในตัวอยู่แล้ว ในขณะที่ K-means จะยัดทุกจุดเข้า

กลุ่มใดกลุ่มหนึ่งเสมอ แม้จะเป็นจุดเดี่ยวโดด ๆ ก็ตาม

หลักการ: 2 พารามิเตอร์ + 3 บทบาทของจุด

DBSCAN ใช้ค่าคงที่ 2 ตัวเท่านั้น:  $\epsilon$  = รัศมีค้นหาเพื่อนบ้าน (โปรเจกต์นี้ตั้ง 400 เมตร) และ

$\min\_samples$  = จำนวนเพื่อนบ้านขั้นต่ำที่ต้องมี (โปรเจกต์นี้ตั้ง 3) จากนั้นไล่ตรวจทุกจุดที่ละจุด

ด้วยกติกาเดียวกัน:

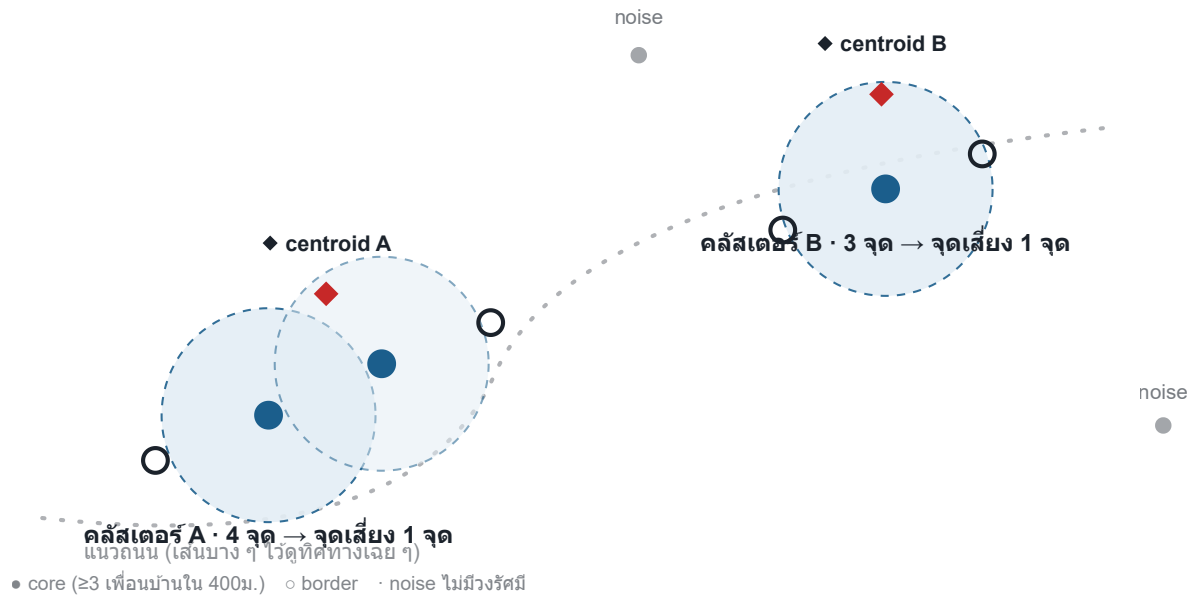
- Core point — จุดที่มีเพื่อนบ้าน (รวมตัวเอง)  $\geq \min\_samples$  ภายในรัศมี 400 เมตร ถือว่าอยู่ในย่านที่ "หนาแน่นพอ"
- Border point — ตัวเองไม่หนาแน่นพอจะเป็น core แต่อยู่ในรัศมี 400 เมตรของ core point จุดอื่น เลยถูกดึงเข้ากลุ่มเดียวกันด้วย (นับเป็นสมาชิก แต่ไม่ขยายกลุ่มต่อ)
- Noise — ไม่ใช่ทั้งสองอย่างข้างต้น อยู่โดดเดี่ยวเกินไป → ได้ label -1 และถูกทิ้ง ไม่นับเป็นจุดเสียงใด ๆ

กลไกสำคัญที่สุดคือ การไล่ลาม (chaining): ถ้า core point A อยู่ในรัศมีของ core point B

และ B อยู่ในรัศมีของ core point C ต่อกันไปเรื่อย ๆ ทั้ง A-B-C จะถูกรวมเป็นกลุ่มเดียวกันหมด

แม้ A กับ C เองจะอยู่ห่างกันเกิน 400 เมตรก็ตาม (เพราะเชื่อมกันผ่าน B) — นี่คือเหตุผลที่จุดเสียง

จริงมักลากยาวตามแนวถนนโค้งได้ ไม่ใช่ช่วงกลมตายตัว



วงประ = รัศมี 400 เมตรรอบ core point (เพื่ออธิบายหลักการเท่านั้น — วงเหล่านี้ไม่เคยถูกวาดบนแผนที่จริง ดูหัวข้อ 03) จุดสีเหลี่ยมขาวกลมตัด = ตำแหน่งเฉลี่ย (centroid) ที่กลายเป็นจุดเดียวซึ่งขึ้นแผนที่แทนทั้งกลุ่ม

โค้ดจริง: cluster\_zones()

cluster\_zones(events, eps\_meters, min\_samples)

รัน DBSCAN จริง

```
coords = np.array([[radians(e["lat"]), radians(e["lng"])] for e in events])
labels = DBSCAN(
    eps=eps_meters / EARTH_RADIUS_M, min_samples=min_samples, metric="haversine"
).fit(coords).labels_
```

`radians(...)` แปลงพิกัดจาก "องศา" (ที่ Excel เก็บ) เป็น "เรเดียน" เพราะสูตร haversine และไลบรารีที่ใช้ต้องการหน่วยเรเดียน — ทำไมต้องใช้ haversine แทนระยะทางเส้นตรงธรรมดา (Euclidean)? เพราะโลกกลม 1 องศาจริง ๆ ไม่เท่ากับระยะทางจริงคงที่เสมอ (ใกล้เส้นศูนย์สูตรกว้างกว่าใกล้ขั้วโลก) ถ้าคำนวณระยะทางแบบเส้นตรงบนค่า lat/long ตรง ๆ ระยะจะบิดเบี้ยวไปตามละติจูด haversine คือสูตรคำนวณระยะทางตามส่วนโค้งบนผิวทรงกลมซึ่งให้ค่าตรงกับระยะจริงบนพื้นโลก

`eps_meters / EARTH_RADIUS_M` คือการแปลง "อยากได้ 400 เมตรจริง" ให้เป็น "มุมกี่เรเดียน" เพราะเมตริก haversine ของ sklearn วัดระยะเป็นมุม ไม่ใช่เมตร ตามสูตร  $\text{ความยาวส่วนโค้ง} = \text{รัศมีโลก} \times \text{มุม (เรเดียน)}$  เมื่อจัดสมการใหม่จะได้  $\text{มุม} = \text{ความยาวส่วนโค้ง} \div \text{รัศมีโลก}$  — บรรทัดนี้คือสมการนั้นตรง ๆ ( $\text{EARTH\_RADIUS\_M} = 6,371,000$  เมตร ค่ารัศมีเฉลี่ยของโลก)

`.fit(coords).labels_` รันอัลกอริทึมตามหลักการด้านบนกับทั้ง 4,460 จุดในคราวเดียว ผลลัพธ์คือ array ตัวเลข 1 ค่าต่อ 1 จุด: -1 แปลว่า noise, ตัวเลข 0,1,2,... คือรหัสกลุ่ม (จุดที่ได้เลขเดียวกัน = อยู่กลุ่มเดียวกัน)

```
for cluster_id in sorted(set(labels)):
    if cluster_id == -1:
        continue
    members = [events[i] for i, l in enumerate(labels) if l == cluster_id]
    ...
    "lat": round(sum(m["lat"] for m in members) / n, 6),
    "lng": round(sum(m["lng"] for m in members) / n, 6),
```

วนรหัสกลุ่มที่ไม่ใช่ -1 แต่ละรหัส ดึงอุบัติเหตุดับทุกแถวที่ได้รับหานั้นมารวมเป็นลิสต์ `members` — นี่คือ "อุบัติเหตุจริงทุกแถวที่ DBSCAN ตัดสินว่าอยู่ย่านเดียวกัน"

บรรทัดที่สำคัญที่สุดของทั้งไฟล์อยู่ตรงนี้: `lat/lng` ของจุดเสี่ยง คือค่าเฉลี่ยเลขคณิตธรรมดาของพิกัดสมาชิกทุกคน (centroid) — พิกัดดับของอุบัติเหตุแต่ละแถวจะหายไปหมด เหลือแค่ 1 ตำแหน่งเฉลี่ยเดียวที่เป็นตัวแทนทั้งกลุ่มไปตลอด ตั้งแต่ตอนคำนวณคะแนนจนถึงตอนขึ้นจุดบนแผนที่ — เก็บไว้คู่กับสถิติรวมของกลุ่ม (จำนวนครั้ง, ผู้เสียชีวิต/บาดเจ็บ, ล้อส่วนรถคันเดียว, ถนนที่พบบ่อยที่สุด ฯลฯ)

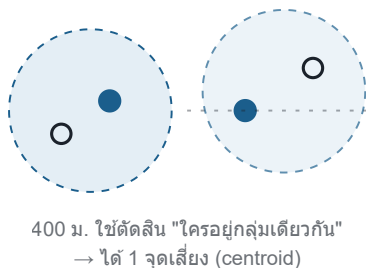
ผลจริงของโปรเจกต์นี้: 4,460 จุดมีพิกัด → DBSCAN(`eps=400m`, `min=3`) → 171 จุดเสี่ยง ครอบคลุม 3,784 เหตุการณ์ · เหลือ 676 เหตุการณ์เป็น noise (กระจัดกระจาย ไม่เข้าเกณฑ์กลุ่มใด ถูกทิ้งจากแผนที่)

### 03 ทำไมจุดบนแผนที่ถึงไม่ติดกัน?

คำตอบสั้นที่สุด: แผนที่ไมเคียวาววงกลมรัศมี 400 เมตรแค่ดวงเดียว เลขนั้นถูกใช้ "ตอนคำนวณ" ใน Python เท่านั้น แล้วก็หายไปจากผลลัพธ์ทั้งหมด

ต้องแยก 2 ขั้นตอนออกจากกันให้ชัดก่อน เพราะเป็นคนละโปรแกรม คนละภาษา คนละเวลารันกันเลย:

#### ตอนคำนวณ (Python, ครั้งเดียวต่อรอบ)



#### ตอนแสดงผล (ทุกครั้งที่เปิดเว็บ)



ซ้าย = สิ่งที่เกิดขึ้นใน scripts/build\_risk\_points.py (มองไม่เห็น รันแค่ตอน calibrate) · ขวา = สิ่งที่ใช้เห็นจริงจาก js/riskpoints.js

js/riskpoints.js → drawOnMap(map)

วาดจุดจริงบนแผนที่

```
const marker = L.circleMarker([p.lat, p.lng], {
  radius: 9,
  color: style.color,
  weight: 2,
  fillColor: style.color,
  fillOpacity: 0.5,
}).addTo(map);
```

L.circleMarker ของ Leaflet (ไลบรารีแผนที่ที่ใช้) คือ "หมุด" รูปวงกลมที่ radius วัดเป็นพิกเซลบนจอ ไม่ใช่เมตรบนโลก — ตั้งไว้ 9 แปลว่า "วาดวงกลมกว้าง 9 พิกเซลเสมอ" ไม่ว่าจะซูมออกจนเห็นทั้งประเทศ หรือซูมเข้าจนเห็นระดับถนน ขนาดบนจอจะเท่าเดิมทุกครั้ง มันคือไอคอนปักหมุดตำแหน่ง ไม่ได้แทนระยะทางจริงใด ๆ ทั้งสิ้น

เทียบกับอีกฟังก์ชันในโปรเจกต์เดียวกัน — js/map.js ใช้ L.circle(latlng, {radius: accuracyM}) วาดวงกลมความแม่นยำ GPS ของตำแหน่งผู้ใช้เอง L.circle (ไม่มีคำว่า Marker) ต่างจากตัวบนตรงที่ radius วัดเป็น เมตรจริง วงกลมจะขยาย/หดตามการซูมจริง ๆ โปรเจกต์นี้ใช้ทั้งสองแบบโดยตั้งใจ แยกหน้าที่กันชัดเจน: L.circle สำหรับสิ่งที่ต้องสเกลตามระยะจริง (ความแม่นยำ GPS) และ L.circleMarker สำหรับหมุดปักตำแหน่งจุดเสียง — ตัวเลข 400 เมตรของ DBSCAN ไม่เคยถูกส่งเข้าฟังก์ชันวาดวงกลมแบบไหนเลยสักที



สรุปว่าทำไมไม่ติดกัน — สองเหตุผลซ้อนกัน:

1. สิ่งที่ว่าคือไอคอนขนาดคงที่ 9 พิกเซล ไม่ใช่รูปทรงตามสเกลจริง จึงไม่มีอะไรที่ "ควรจะ" ชนกันตามระยะทางจริงอยู่แล้ว
2. ต่อให้ย่อภาวดาวรัศมี 400 เมตรจริง ๆ — โดยนิยามของ DBSCAN เอง สองจุดเสี่ยงที่แยกกันคนละกลุ่มคือกลุ่มที่ "ไม่มีทางไต่ลามถึงกันได้ภายในระยะ 400 เมตรต่อครั้ง" ดังนั้นระยะจริงระหว่าง centroid ของสองจุดเสี่ยงคนละกลุ่มจะมากกว่า 400 เมตรเสมอ (มักจะมากกว่านั้นเยอะ เพราะ centroid คือค่าเฉลี่ย ไม่ใช่ขอบกลุ่ม) แม้แต่วงกลม 400 เมตรจริง ๆ ก็แทบไม่มีทางชนกันด้วยตัวมันเอง

## 04 กายภาพถนน — Geometric Complexity

2 ฟังก์ชัน: `conflict_group`, `compute_fi_weights` — ให้นำหนักตามรูปแบบ

ถนนจากข้อมูลอุบัติเหตุจริงในไฟล์เอง ไม่ยืมตัวเลขต่างประเทศ

`conflict_group(location_type)`

จัดกลุ่มถนน 4 แบบ

```
if "ต่างระดับ" in t or "ramp" in t.lower(): return "diverging", True
if "สามแยก" in t or "สี่แยก" in t: return "crossing", True
if "โค้ง" in t or "ทางร่วม" in t or "เชื่อม" in t: return "merging", True
if "ทางตรง" in t: return "ncp", True
return "ncp", False
```

อ่านคอลัมน์ "บริเวณที่เกิดเหตุ" ของแต่ละแถว จับคำสำคัญภาษาไทยเพื่อจัดเข้ากลุ่มลักษณะถนน 4 แบบ: ncp (ทางตรง ไม่มีจุดตัดกระแสจราจร — ใช้เป็นค่าฐาน), merging (ทางโค้ง/ทางร่วม/ทางเชื่อม), diverging (ทางต่างระดับ/ramp), crossing (สามแยก/สี่แยก)

ลำดับการเช็ค สำคัญมาก: ต้องเช็ค "ต่างระดับ" ก่อน "แยก" เพราะข้อความบางแบบอาจมีทั้งสองคำปนกัน (เช่น "ทางแยกต่างระดับ") ถ้าเช็คผิดลำดับจะจัดกลุ่มผิด — โค้ดจึงเรียง diverging → crossing → merging → ncp เป็นลำดับตายตัว ข้อความที่จับคำไม่เข้าเงื่อนไขไหนเลย (เช่น "อื่นๆ" หรือคำว่าว่าง) จะ fallback เป็น ("ncp", False) — ได้กลุ่ม ncp (ปลอดภัยไว้ก่อน) แต่ค่า False ตัวที่สองจะกันไม่ให้แถวนี้ถูกนับตอนประมาณค่า FI Rate ด้านล่าง เพราะมันไม่ใช่ข้อมูลจริงของกลุ่มใดกลุ่มหนึ่ง

```
if e["deaths"] > 0 or e["serious"] > 0 or e["minor"] > 0:
    tally[group]["fi"] += 1
...
rate = (t["fi"] / t["n"]) if t["n"] else base_rate
weights[g] = round(rate / base_rate, 3)
```

FI ย่อจาก Fatal-or-Injury คือแหวไหนก็ตามที่มีผู้เสียชีวิตหรือบาดเจ็บอย่างน้อย 1 คน (ไม่ว่าระดับใด) นับเป็น FI สูตรคือ: FI Rate ของกลุ่ม = จำนวนเหตุที่เป็น FI ÷ จำนวนเหตุทั้งหมดของกลุ่มนั้น แล้ว น้ำหนัก = FI Rate ของกลุ่มนั้น ÷ FI Rate ของกลุ่ม ncp (ncp คือเส้นฐาน weight เท่ากับ 1.000 เสมอโดยนิยาม) พุดง่าย ๆ คือ "ถนนแบบนี้มีโอกาสเกิดอุบัติเหตุถึงขั้นบาดเจ็บ/ตายมากกว่าถนนตรงธรรมดาแค่ไหน"

| กลุ่ม CONFLICT                 | N (เหตุการณ์) | FI RATE | น้ำหนัก |
|--------------------------------|---------------|---------|---------|
| NCP — ทางตรง (ฐาน)             | 3,948         | 42.1%   | 1.000   |
| Merging — โโค้ง/ทางร่วม/เชื่อม | 230           | 42.6%   | 1.011   |
| Diverging — ต่างระดับ/ramp     | 54            | 35.2%   | 0.835   |
| Crossing — สามแยก/สี่แยก       | 35            | 57.1%   | 1.356   |

ตัวเลขจริงบอกว่า ทางแยก (crossing) มีโอกาสเกิดเหตุถึงขั้นบาดเจ็บ/เสียชีวิตสูงกว่าทางตรงถึง 35.6% ซึ่งตรงกับหลักวิศวกรรมจราจรทั่วไปที่ว่าจุดตัดกระแส (crossing) รุนแรงกว่าจุดรวม/แยกกระแส (merging/diverging) ส่วน diverging ที่ได้ 0.835 (ต่ำกว่าฐาน) มีตัวอย่างแค่ 54 เหตุการณ์ — น้อยพอที่เอกสารอ้างอิงของโปรเจกต์เตือนไว้ว่าต้องตีความอย่างระมัดระวัง

ค่า gc\_score ของแต่ละจุดเสี่ยง = ค่าเฉลี่ยน้ำหนักของทุกอุบัติเหตุในจุดนั้น (คำนวณในฟังก์ชัน score\_zones หัวข้อถัดไป)

05 สูตร 4 เกณฑ์คะแนน (25% เท่ากัน)

ทุกเกณฑ์แปลงเป็นสเกล 0–100 ด้วยวิธีเดียวกันก่อน คือ Percentile Rank แล้วค่อยถ่วงน้ำหนักรวมกัน

ก่อนอื่น: Percentile Rank คืออะไร

percentile\_rank(series)

normalize ทุกเกณฑ์

return series.rank(pct=True, method="average") \* 100

ตัวอย่างจากชุดทดสอบจริงในไฟล์ ค่าดิบ [10, 20, 20, 30] — เรียงลำดับได้อันดับ 1,2,3,4 แต่ค่า 20 ซ้ำกัน 2 ตัว จึงเฉลี่ยอันดับที่ 2 กับ 3 เป็น 2.5 ให้ทั้งคู่เท่ากัน (method="average" คือการจัดการ "ค่าเสมอกัน" แบบนี้) ได้อันดับ [1, 2.5, 2.5, 4] หารด้วยจำนวนทั้งหมด (4) แล้วคูณ 100 → [25, 62.5, 62.5, 100]

ความหมาย: ถ้าจุดหนึ่งได้ percentile rank ของ "ความถี่" เท่ากับ 90 แปลว่า 90% ของจุดเสี่ยงทั้งหมดในรอบนี้มีความถี่อุบัติเหตุน้อยกว่าหรือเท่ากับจุดนี้ — เป็นค่าเปรียบเทียบกับ "ตำแหน่งสัมพัทธ์" ในกลุ่ม ไม่ใช่ค่าความเสี่ยงสมบูรณ์

ทำไมใช้วิธีนี้แทน min-max หรือ log-ratio (สูตรเดิมของโมเดล v1): ทดสอบจริงโดยเติมจุดปลอม (outlier) ที่มี n=100 อุบัติเหตุเข้าไปในชุดข้อมูล แล้ววัดว่าคะแนนของจุดปกติที่ freq=10 เปลี่ยนไปเท่าไรที่ตัวมันเองไม่ได้เปลี่ยนอะไรเลย — ด้วย Percentile Rank คะแนนขยับแค่ 0.3 แต้ม (95.0→94.7) แต่ด้วยสูตร log-ratio เดิมคะแนนขยับถึง 6.76 แต้ม (22.35→15.59) มากกว่ากันเกือบ 23 เท่า Percentile Rank จึงเสถียรกว่ามาก ไม่ไวต่อค่าผิดปกติ ทำให้เทียบคะแนนข้ามช่วงเวลาได้อย่างมีความหมาย

เกณฑ์ที่ 1 — ความถี่อุบัติเหตุ

Frequency\_Score(จุด) = PercentileRank( accident\_count ของจุดนั้น )

accident\_count      จำนวนอุบัติเหตุดิบที่ DBSCAN รวมเข้าจุดนี้ (ตัวแปร n ในโค้ด)

สัญญาณตรงไปตรงมาที่สุด — เกิดบ่อยที่จุดเดิมซ้ำ ๆ ย่อมเสี่ยงกว่า อ้างอิงเชิงคุณภาพ (ไม่ใช่ส่วนหนึ่งของสูตร): จุดที่มีเหตุบาดเจ็บ/เสียชีวิต ≥3 ครั้ง/ปี เข้านิยาม "Black Spot" ของออสเตรเลีย ใช้เทียบผลลัพธ์เฉย ๆ

เกณฑ์ที่ 2 — มูลค่าความเสียหายทางเศรษฐกิจ

economic\_loss(จุด) = deaths×6,700,000 + serious×2,000,000 + minor×58,000    (บาท)

EconomicLoss\_Score(จุด) = PercentileRank( economic\_loss )

deaths/serious/minor      ผลรวมผู้เสียชีวิต/บาดเจ็บสาหัส/บาดเจ็บเล็กน้อยของทุกอุบัติเหตุในจุดนั้น

6.7M / 2M / 58k      ต้นทุนความสูญเสียเฉลี่ยต่อราย (บาท) จาก TDRI ปีงบประมาณ 2565

นับจำนวนอย่างเดียวนี้อย่างเหมือนเกณฑ์ 1 ปฏิบัติกับอุบัติเหตุเล็กน้อยและเหตุถึงชีวิตเท่ากันหมด เกณฑ์นี้ถึง "ความรุนแรง" เข้ามาถ่วงด้วยมูลค่าความเสียหายจริงเป็นบาท ให้จุดที่เคยมีคนตายโดดเด่นกว่าจุดที่มีแต่รถเฉี่ยวชนเล็กน้อยแม้จำนวนครั้งจะพอ ๆ กัน

### เกณฑ์ที่ 3 — สัดส่วนรถคันเดียว (Single-Vehicle Ratio)

$$\text{Single-Vehicle \% (จุด)} = (\text{เหตุที่ "รถที่เกิดเหตุ" } \leq 1) \div \text{เหตุทั้งหมดของจุด} \times 100$$
$$\text{SingleVehicle\_Score(จุด)} = \text{PercentileRank}(\text{Single-Vehicle \%})$$

**vehicles**      จำนวนคันรถที่เกี่ยวข้องต่ออุบัติเหตุ 1 แถว

รถคันเดียวสูง (หลุดโค้ง, พลิกคว่ำ, ชนวัตถุข้างทาง) มักชี้ปัญหา "กายภาพถนน/ไหล่ทาง" ส่วนรถหลายคันสูงชี้ปัญหา "จุดขัดแย้งจราจร/ทางแยก" — ค่า single\_pct กับ multi\_pct รวมกันเป็น 100% เสมอ (สหสัมพันธ์ -1.00 เป๊ะ) ใส่ทั้งคู่ในสูตรจะไม่เพิ่มข้อมูลใหม่เลย จึงใช้แค่ single\_pct ในการคิดคะแนน แต่ยังโชว์ทั้งคู่ในหน้าเว็บเพื่อวินิจฉัยรูปแบบปัญหา (อ้างอิง FHWA Roadway Departure Crashes สำหรับคำแนะนำเชิงวิศวกรรมที่ตามมา เช่น เส้นสั่นเตือนขอบทาง)

### เกณฑ์ที่ 4 — Geometric Complexity

$$\text{gc\_score(จุด)} = \text{ค่าเฉลี่ยน้ำหนัก FI ของทุกอุบัติเหตุในจุดนั้น}$$
$$\text{GeometricComplexity\_Score(จุด)} = \text{PercentileRank}(\text{gc\_score})$$

น้ำหนักมาจากหัวข้อ 04 (1.000 / 1.011 / 0.835 / 1.356 ตามกลุ่มถนน) — จุดที่อุบัติเหตุส่วนใหญ่เกิดแถวทางแยกจะมีค่าเฉลี่ยสูงกว่าจุดที่ส่วนใหญ่เป็นทางตรง แล้วเทียบ percentile กับจุดอื่นอีกที

### สูตรรวมคะแนนสุดท้าย

$$\begin{aligned} \text{Risk Score(จุด)} = & 0.25 \times \text{Frequency\_Score} \\ & + 0.25 \times \text{EconomicLoss\_Score} \\ & + 0.25 \times \text{SingleVehicle\_Score} \\ & + 0.25 \times \text{GeometricComplexity\_Score} \end{aligned}$$

ผลลัพธ์อยู่ในช่วง 0–100 เสมอ (เพราะเป็นค่าเฉลี่ยถ่วงน้ำหนักของ percentile 0–100 สี่ตัว) ทำให้ถ่วงเท่ากันหมด 25%: ทดสอบด้วย PCA (Principal Component Analysis) พบว่าองค์ประกอบหลักตัวแรกอธิบายความแปรปรวนได้แค่ราว 40% เท่านั้น — ไม่มีปัจจัยรวมเด่นตัวเดียวที่ครอบงำ แกมสองเกณฑ์ (รถคันเดียว, กายภาพถนน) ยังมีค่า loading ติดลบด้วยซ้ำ และเมื่อเช็คสหสัมพันธ์แบบ Spearman ทุกคู่เกณฑ์ ก็ต่ำกว่า 0.30 ทั้งหมด (แปลว่าทั้ง 4 เกณฑ์ค่อนข้างเป็นอิสระต่อกันจริง ต่างคนต่างให้ข้อมูลคนละมุม) เมื่อไม่มีเหตุผลทางทฤษฎีหรือสถิติให้ถ่วงน้ำหนัก เกณฑ์ใดมากกว่ากัน คู่มือ OECD/JRC แนะนำให้ใช้ "equal weighting" เป็นค่าเริ่มต้นที่ปลอดภัยที่สุด — โปรเจกต์นี้จึงเลือก 25/25/25/25 อย่างมีเหตุผลรองรับ ไม่ใช่การเดาสุ่ม

score\_zones(zones, fi\_weights)

ประกอบทุกสูตรเข้าด้วยกัน

```
df = pd.DataFrame(zones)
df["s_freq"] = percentile_rank(df["accident_count"])
df["s_econ"] = percentile_rank(df["economic_loss"])
df["s_single"] = percentile_rank(df["single_pct"])
df["s_geom"] = percentile_rank(df["gc_score"])
df["risk_score"] = (0.25*df.s_freq + 0.25*df.s_econ
                    + 0.25*df.s_single + 0.25*df.s_geom).round(1)
```

เอาทั้ง 171 จุดเสี่ยงมาใส่ในตารางเดียว (DataFrame) แล้วเรียก percentile\_rank ที่เดียวทั้งคอลัมน์ — นี่คือเหตุผลที่ Percentile Rank เป็น "ค่าเทียบสัมพัทธ์เฉพาะรอบนี้": มันคำนวณโดยเทียบกับ 171 แถวในตารางนี้พร้อมกันทั้งหมด ถ้าจำนวนหรือองค์ประกอบของจุดเสี่ยงเปลี่ยน (รอบ calibration ถัดไป) ตัวเลข percentile ของจุดเดิมก็จะขยับตามไปด้วย นี่คือที่มาของกฎ "ห้ามคำนวณ percentile ใหม่ทุก request" — ต้องล็อกเป็นเวอร์ชันตายตัวต่อรอบ ไม่งั้นคะแนนจะ "ลอย" ขึ้นลงทุกครั้งที่มีอุบัติเหตุใหม่เข้ามาแม้เหตุเดียว

## 06 แบ่งระดับ ต่ำ/กลาง/สูง ด้วย Jenks Natural Breaks

ทำไมไม่ตัดตายตัวที่ 0-33 / 33-66 / 66-100? เพราะการตัดแบบเปอร์เซ็นต์คงที่ไม่สนใจ "รูปร่างจริง" ของข้อมูลเลย อาจตัดกลางกลุ่มก่อนเดียวกันจนสองจุดที่คะแนนใกล้เคียงกัน มากถูกแยกคนละระดับ

Jenks คือการหาจุดตัดที่ทำให้ ความแปรปรวนภายในแต่ละกลุ่มน้อยที่สุด (แต่ละกลุ่มมีความสม่ำเสมอในตัวเองมากที่สุด) ซึ่งเท่ากับทำให้ความแปรปรวน ระหว่าง กลุ่มมากที่สุดไปโดยปริยาย เป็นวิธีเดียวกับที่ CDC (ศูนย์ควบคุมโรคสหรัฐฯ) ใช้ทำแผนที่จุดร้อน (hotspot mapping) จริง

jenks\_breaks(values, n\_classes=3)

Fisher's optimal partition (DP)

```
def sse(i, j): # ความแปรปรวนรวมของช่วง v[i:j]
    m = j - i; s = csum[j] - csum[i]
    return (csum2[j] - csum2[i]) - s * s / m

dp[c][j] = min over i of ( dp[c-1][i] + sse(i, j) )
```

csum/csum2 คือผลรวมสะสมของค่าและค่ายกกำลังสอง ทำให้คำนวณ "ผลรวมความคลาดเคลื่อนกำลังสอง (SSE)" ของช่วงข้อมูลใด ๆ ได้ในขั้นตอนเดียว ด้วยสูตร  $SSE = \sum x^2 - (\sum x)^2 / n$  แทนที่จะต้องไล่หาค่าเฉลี่ยแล้วบวกใหม่ทุกครั้ง (ทำให้เร็วขึ้นมาก)

dp[c][j] คือ "SSE รวมที่ต่ำที่สุดเท่าที่เป็นไปได้ ถ้าแบ่งข้อมูล j ตัวแรก (ที่เรียงแล้ว) ออกเป็น c กลุ่ม" — โค้ดลองทุกตำแหน่งตัดที่เป็นไปได้ แล้วเก็บเฉพาะตำแหน่งที่ให้ผลรวม SSE ต่ำสุดไว้ใน cut[c][j] เพื่อย้อนกลับไปหาตำแหน่งตัดจริงได้ที่หลัง นี่คือเทคนิค Dynamic Programming — รับประกันว่าได้จุดตัดที่ดีที่สุดจริง ๆ ไม่ใช่แค่ค่าประมาณ

ผลลัพธ์คือลิสต์จุดตัด เช่นชุดทดสอบ [1, 2, 3, 10, 11, 12, 100, 101, 102] แบ่ง 3 กลุ่มจะได้ [3.0, 12.0] พอดี (กลุ่มแรกจบที่ 3, กลุ่มสองจบที่ 12, ที่เหลือคือกลุ่มสาม) — เห็นชัดว่ามันหา "ช่องว่างธรรมชาติ" ของข้อมูลเจอจริง ไม่ใช่หารตัวเลขหนึ่ง ๆ

ผลจริงของรอบ v2568-r2

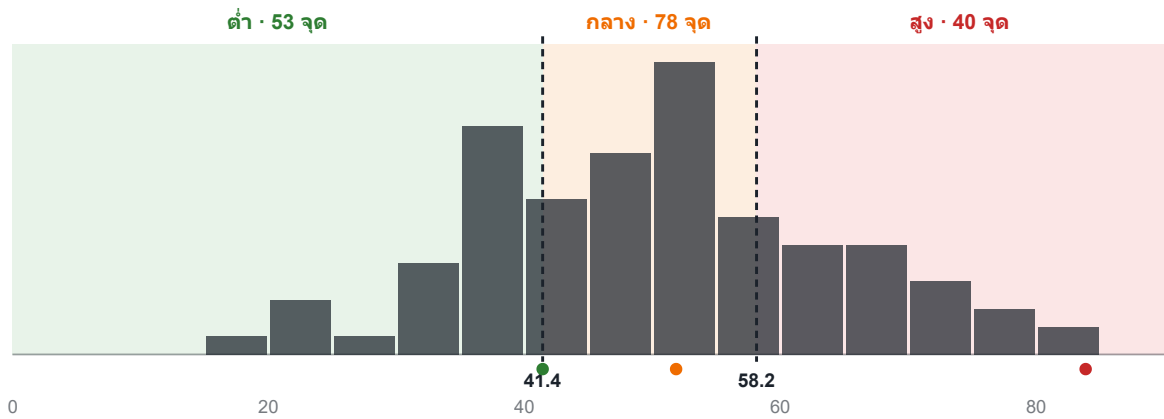
classify(score, breaks)

ตัดสินระดับ

```
if score ≤ breaks[0]: return "low"
if score ≤ breaks[1]: return "medium"
return "high"
```

ใช่ ≤ ไม่ใช่ < — คะแนนที่ตรงเป๊ะกับจุดตัดจะถูกจัดเข้ากลุ่ม "ต่ำกว่า" เสมอ (ดูตัวอย่างจริงด้านล่าง มีจุดหนึ่งชนเส้นแบ่งพอดี)

| ระดับ     | ช่วง RISK SCORE                 | จำนวนจุด |
|-----------|---------------------------------|----------|
| ● ต่ำ     | คะแนน $\leq 41.4$               | 53       |
| ▲ ปานกลาง | $41.4 < \text{คะแนน} \leq 58.2$ | 78       |
| ■ สูง     | คะแนน $> 58.2$                  | 40       |



ฮิสโตแกรมคะแนนจริงของ 171 จุด (ช่วงกว้าง 5 คะแนนต่อแห่ง) จุดกลมเล็กบนเส้นแกน = ตำแหน่งของ 3 ตัวอย่างจริงด้านล่าง (41.4 / 51.9 / 83.9)

### ตัวอย่างจริงจากข้อมูล 3 ระดับ

สะพานภูมิพลที่ 1 ไปสะพานภูมิพลที่ 2  
สมุทรปราการ

83.9 /100

■ สูง

156

ครั้ง

9

เสียชีวิต

2

สาหัส

100

เล็กน้อย

ความถี่      98.2  
ความเสียหาย      98.2  
รถคันเดียว      41.5  
กายภาพถนน      97.7

$0.25 \times (98.2 + 98.2 + 41.5 + 97.7) = 83.9$  · ความเสียหายรวม 70.1  
ล้านบาท

สะพานข้ามแม่น้ำท่าจีนฝั่งตะวันตก-นาโคก  
สมุทรสาคร

51.9 /100

▲ ปานกลาง

17

ครั้ง

0

เสียชีวิต

0

สาหัส

10

เล็กน้อย

ความถี่      87.7  
ความเสียหาย      55.3  
รถคันเดียว      24.0  
กายภาพถนน      40.6

$0.25 \times (87.7 + 55.3 + 24.0 + 40.6) = 51.9$  · ความเสียหายรวม 5.8  
ล้านบาท



บางบอน-โรงพยาบาลบางปะกอก8

กรุงเทพมหานคร

41.4/100

● ต่ำ

4

ครั้ง

0

เสียชีวิต

0

สาเหตุ

6

เล็กน้อย

ความถี่ 41.8

ความเสียหาย 50.0

รถคันเดียว 33.0

กายภาพถนน 40.6

$0.25 \times (41.8 + 50.0 + 33.0 + 40.6) = 41.35 \rightarrow$  บัด 41.4 · ขนเส้น  
แบ่งพอดี  $\rightarrow$  ต่ำ (ใช้  $\leq$ )

จุดตัวอย่างที่ 3 คะแนนออกมาตรงกับ `breaks[0]` (41.4) เป๊ะ — เพราะ `classify()` เช็ค `score ≤ breaks[0]` จุดนี้เลยถูกจัดเป็น "ต่ำ" ไม่ใช่ "ปานกลาง" นี่คือตัวอย่างจริงของกติกาก่อนที่เส้นแบ่งเป๊ะ ๆ

## ๐7 บันทึกผลลัพธ์ + self-test

to\_geojson(zones, calibration)

แปลงเป็นไฟล์แผนที่

```
{
  "type": "FeatureCollection",
  "calibration": calibration,
  "features": [{ "geometry": { "coordinates": [z.lng, z.lat]}, ... }]
}
```

มาตรฐาน GeoJSON กำหนดลำดับพิกัดเป็น [ลองจิจูด, ละติจูด] — สลับจากที่คนทั่วไปพูดกันว่า "lat, lng" จึงต้องระวังจุดนี้เป็นพิเศษเวลาอ่าน/เขียนไฟล์นี้ ส่วนคีย์ "calibration" ที่ปะไว้ข้าง ๆ "type"/"features" ไม่ใช่มาตรฐาน GeoJSON ปกติ แต่เป็น "foreign member" ที่ผู้อ่านทั่วไป (เช่น `fetch().json()` ฝั่งเว็บ) จะเก็บไว้เฉย ๆ โดยไม่ฟัง — ใช้อธิบายว่าไฟล์นี้คำนวณจากรอบ calibration เวอร์ชันไหน

main()

ประกอบทุกขั้นตอน

เรียงงานตามลำดับ: อ่าน Excel → คำนวณน้ำหนักกายภาพถนนจากทุกแถวไม่ว่าจะมีพิกัดหรือไม่ (เพราะความรุนแรงไม่ต้องพึ่งพิกัด) → กรองเหลือแถวที่มีพิกัดใช้ได้ → รัน DBSCAN → ให้คะแนน 4 เกณฑ์ + แบ่งระดับ Jenks → พิมพ์สรุป (top 5 จุดเสี่ยงคะแนนสูงสุด, จุดตัด Jenks, จำนวนแต่ละระดับ) → ประกอบ dict `calibration` เก็บทุกพารามิเตอร์ที่ใช้ (eps/min\_samples, น้ำหนัก 4 เกณฑ์, ต้นทุนบาท 3 ระดับ, ตาราง FI weight, จุดตัด Jenks, จำนวนจุด/ระดับ, เวลาที่รัน) → บันทึกไฟล์หลัก `risk_points_bkk_metro.geojson` และบันทึกสำเนาการอีกชุดไว้ที่ `data/calibrations/v2568-r2.json` — ไฟล์สำเนาเป็นหลักฐานย้อนหลังว่ารอบนี้คำนวณด้วยพารามิเตอร์อะไร แม้รอบถัดไปจะเขียนทับไฟล์หลักแล้วก็ตาม

\_self\_test() --selftest

ตรวจสอบสูตรแบบเร็ว

ชุด `assert` ตรวจสอบว่าฟังก์ชันคณิตศาสตร์หลักยังถูกต้องอยู่ โดยไม่ต้องเปิดไฟล์ Excel จริงหรือรอ DBSCAN คำนวณ 4,460 จุด (เสร็จในเสี้ยววินาที) ครอบคลุม: ลำดับความสำคัญของ `conflict_group`, เลขคณิตของ tie-breaking ใน `percentile_rank`, ความถูกต้องของ `jenks_breaks` บนชุดตัวอย่างที่มี 3 กลุ่มชัดเจน, พฤติกรรมขอบเขตของ `classify`, ผลรวมค่าคงที่บาท ( $6.7M + 2M + 58k = 8,758,000$ ), การคำนวณ FI weight บนตัวอย่างจำลอง, และการเว้นค่า "ไม่ระบุ" ของ `mode_of` — มีประโยชน์มากเวลาแก้โค้ดแล้วอยากรู้เร็ว ๆ ว่าสูตรยังพังไหม

## 08 ข้อจำกัดที่ควรรู้

- 1 เกณฑ์ Diverging มีตัวอย่างแค่ 54 เหตุการณ์ — น้ำหนัก 0.835 มีความไม่แน่นอนสูงกว่ากลุ่มอื่น
- 2 167 เหตุการณ์ไม่มีพิกัด (3.6%) ถูกตัดจากการจับกลุ่ม/แผนที่ แต่ยังใช้คำนวณ FI Rate ได้เพราะความรุนแรงไม่ต้องอาศัยตำแหน่ง
- 3 676 เหตุการณ์เป็น noise ตามธรรมชาติของ DBSCAN — ไม่ใช่ข้อผิดพลาด แต่ควรรายงานตัวเลขนี้คู่กันเสมอ
- 4 "รถที่เกิดเหตุ" นับจำนวนคันอย่างเดียว ไม่แยกประเภทคู่กรณี (รถ-คนเดินเท้า ก็นับเป็น 1 คัน จึงเข้ากลุ่ม Single)
- 5 Percentile Rank เป็นคะแนน "เชิงเปรียบเทียบภายในรอบ" — 90 แปลว่าอยู่ 10% แรกของรอบนั้น ไม่ใช่ค่าความเสี่ยงสัมบูรณ์ข้ามรอบ

## 09 อ้างอิงหลักการ

- 1 thaincd.com (กรมควบคุมโรค) — นิยาม Black Spot ออสเตรเลีย ( $\geq 3$  ครั้ง/ปี)
- 2 TDRI — ความสูญเสียทางเศรษฐกิจอุบัติเหตุทางถนนไทย ปีงบประมาณ 2565 (ต้นทุน 6.7M/2M/58k บาท)
- 3 FHWA — Roadway Departure Crashes (มาตรการรถคันเดียวหลุดโค้ง)
- 4 TxDOT Design Manual §11.3.7.1 — Conflict Points
- 5 VDOT — ลำดับความรุนแรง crossing > merging/diverging
- 6 ITRE, NC State University (2020) — Weighted Conflict Points (VJuST)
- 7 Nardo et al. (2008), OECD/JRC Handbook — Ranking normalisation + Equal weighting
- 8 Jenks, G.F. (1967) — The Data Model Concept in Statistical Mapping (ใช้โดย CDC)
- 9 Srinivasan & Carter (2011), UNC HSRC/NCDOT — Fixed-Schedule Recalibration
- 10 MOT Data Catalog 2568 — data/accident2025\_1.xlsx (6 จังหวัด, 4,627 เหตุการณ์)

